

PROJECT GUIDELINES

VICINIA

A Campus-Scoped, Location-First Social App · CS409 Mobile Computing (React Native)

Course & Section: CS409 – CS21S2 (BSCS)	Instructor: Sir Juvert De Los Reyes
Leader: Member 1 (you)	Estimated Timeline: 4 to 5 weeks — exact dates set by the Leader
Team: 5 members — Leader + Members 2–5	Teams: Frontend (2) · Backend (2) · Docs owned within Backend

A note on names: since several of you are also on the CS 003 Space Invaders team, this guide never prints real names — only **Member 1** (Leader, you) through **Member 5**. Assign the actual people privately; swap them around freely, the structure doesn't change. The accent color here is teal / amber / plum on purpose, so this document is never confused with the blue Space Invaders one at a glance.

1. What Are We Building?

We're building **Vicinia**, a location-first social app for the T.I.P. QC (and Manila) campus community, in React Native with Firebase as the backend. Instead of a global, follower-driven feed, Vicinia treats *where you physically are* as the main way to discover people and activity: an **Explore map** surfaces what's happening nearby, campus-scoped **Community Boards** group students by interest and radius (Anime & Cosplay, Mobile Dev & Tech, Esports, Campus Life, Buy & Sell, and more), and a **location-adaptive Feed** shifts naturally as you move between the library, a lab building, or a nearby café.

This guide turns your Prelim/Midterm proposal and the current Figma prototype into a week-by-week, person-by-person build plan — the same idea as the guideline format you were given as a reference, just recolored and restructured around Vicinia's own features so the two documents are never mistaken for one another.

Target users: T.I.P. students & faculty, special-interest groups and hobbyists, campus event organizers and recognized student organizations (RSOs), and nearby establishments around the QC/Manila campuses.

2. Key Terms You'll Run Into

A quick reference — flip back here any time a word doesn't ring a bell.

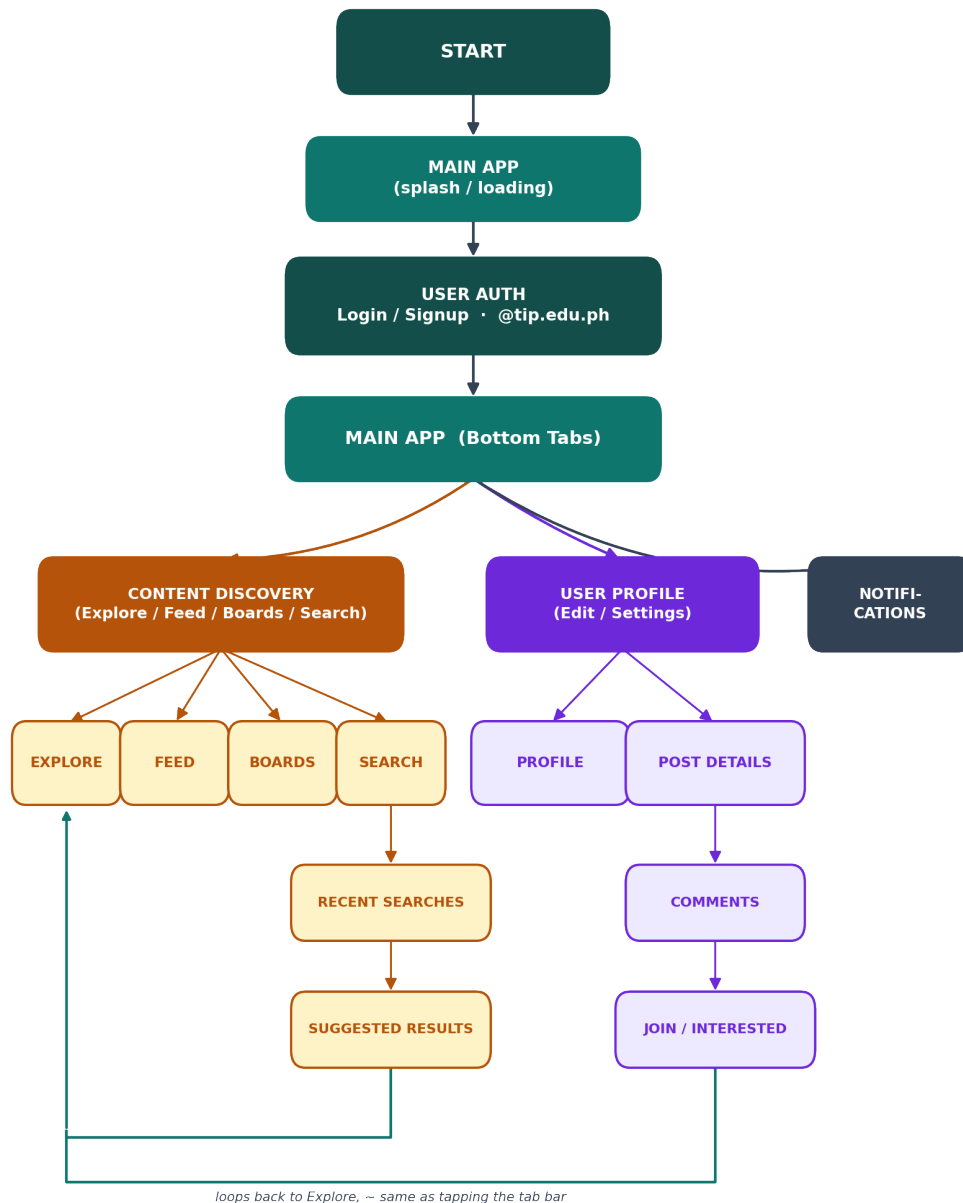
- **Component** — a reusable, self-contained building block of the UI (a post card, a map pin, a button) that renders itself from the props and state it's given.
- **Props / State** — Props are data passed into a component from its parent (read-only). State is data a component owns and can change over time — e.g. which filter tab is currently active.
- **JSX** — the HTML-like syntax inside React Native files that gets compiled into real native UI elements.
- **React Navigation (Stack + Bottom Tabs)** — the library that moves users between screens: a Stack Navigator pushes/pops full screens (Feed → Post Details), while a Bottom Tab Navigator switches between the four main tabs (Explore, Feed, Boards, Profile).
- **Context API + Hooks** — React's built-in way to share data (the logged-in user, active filters) across many screens without manually passing props down through every layer.
- **Firebase / BaaS (Backend-as-a-Service)** — a hosted backend from Google that gives the team a database, login system, and file storage without writing or deploying a custom server.
- **Cloud Firestore** — Firebase's real-time, NoSQL database. When someone posts, comments, or joins a board, everyone else's screen updates automatically — no manual refresh.
- **Firebase Authentication** — the login/signup system; restricting it to @t.i.p.edu.ph addresses is how the app verifies someone is really a T.I.P. student.
- **Geohashing / GeoFirestore** — a technique for indexing locations so "what's near me" queries stay fast, instead of checking the distance to every single event one by one.

- **FlatList** — React Native's component for rendering long scrollable lists (the Feed, a board's posts) efficiently, since it only renders what's currently on screen.
- **Async/await & API calls** — the pattern used to read or write Firebase data without freezing the UI while the network request is in flight.
- **Pull Request (PR) / Merge Conflict** — a PR is a proposed set of changes teammates review before it merges into `main`; a merge conflict happens when two people edit the same lines and Git can't auto-combine them.

3. How the App Works, Step by Step

Every session follows the same shape: a student opens the app, is routed through login (or straight into the app if already signed in), and lands on the four bottom tabs. From there, two natural paths cover almost everything Vicinia does — **discovering** content (Explore / Feed / Boards / Search) and managing **your own presence** (Profile, and anything you've posted or joined) — with Notifications sitting off to the side, reachable from anywhere.

VICINIA — APP FLOW



Redrawn from the team's own Figma app-flow chart, in the guideline's own palette.

The current state of the app (logged-out vs. logged-in) gates what the Bottom Tabs actually show — a logged-out user only ever sees the Auth screen. Once inside, tapping a map pin or a Feed post always resolves to the same **Post Details** screen, whether it came from Explore, Feed, Boards, or Search — that's one screen the whole front-end team should agree on early, since almost every path in the app ends there.

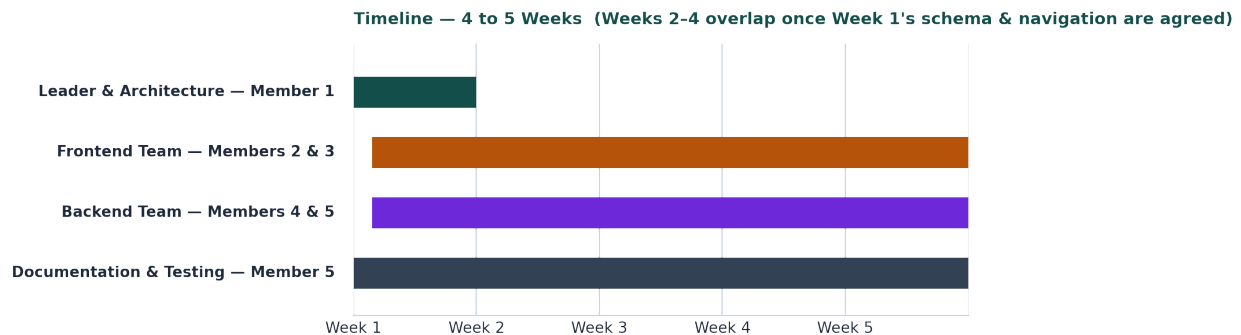
4. Team Roles

Role	Owner	Responsibility
Leader & Core Architecture	Member 1	Owns the Firestore data-schema decisions (users, posts, boards, events, comments) and the navigation architecture so both teams build against the same interface; runs weekly check-ins; merges feature branches into main; has final say on scope calls (e.g. cutting a stretch goal if the team is behind); represents the group to Sir De Los Reyes.
Frontend Team Head FRONTEND	Member 2	Owns the React Navigation setup (Stack + Bottom Tabs), the shared component library (cards, buttons, chips, inputs), and the Explore map + Search screens; coordinates the front-end pair's task split each week.
Frontend — Feed, Boards & Profile FRONTEND	Member 3	Builds the Feed, Boards, Post Details, Profile, and Notifications screens; implements Context API state for the logged-in user and active filters; keeps styling consistent with the Figma file (colors, spacing, 44px+ touch targets).
Backend Team Head BACKEND	Member 4	Owns Firebase Authentication (including the @tip.edu.ph restriction and verified-guest path) and the Cloud Firestore schema/security rules for users, boards, and events; coordinates the back-end pair's task split each week.
Backend + Documentation, Testing & Git Hygiene BACKEND + DOCS	Member 5	Builds the posts/comments data layer and Firebase Cloud Storage (image uploads for posts and profiles); also owns the weekly written reports for Sir De Los Reyes, the README, a test-case checklist, and repo hygiene (issue labels, PR reminders) — compiled from what the whole team did that week, not written solo.

Roles are primary ownership, not silos. Pair up for review — especially Leader ↔ Frontend and Leader ↔ Backend — since Member 1's schema and navigation decisions are what both teams build directly on top of. Everyone can (and should) touch any part of the app when it makes sense; these are default owners, not walls. This also keeps responsibilities roughly equal: every member ships real features, and Member 5's documentation work sits alongside their coding, not instead of it.

5. Timeline (4 to 5 Weeks)

No fixed calendar dates here on purpose — the Leader sets the actual start/end date for each week. Weeks 2 through 4 overlap intentionally, once Week 1's schema and navigation skeleton are agreed on.



Week	Focus	Key Deliverables
1	Setup & Foundations	GitHub repo created with branch/PR conventions; whole team completes a GitHub Skills "Introduction to GitHub" walkthrough (first time for everyone); React Native project running on every device; Firebase project created (Auth, Firestore, Storage enabled); Firestore schema agreed on paper; navigation skeleton stubbed; Figma reviewed and design tokens locked in; feature ownership assigned within each team.
2	Auth + Core Screens	Firebase Authentication wired end-to-end (@tip.edu.ph + verified guest); Context API session state live; all four bottom tabs rendering with real navigation (placeholder data is fine); "users" and "boards" Firestore collections live; first demo of the login → main app flow.
3	Full Loop: Explore, Feed, Boards	React Native Maps integrated with real Firestore pins; nearby-event geo-query working; Feed rendering real posts via FlatList with pull-to-refresh; Boards listing real communities with join/member counts; Search with recent searches + suggested results; first full-team playtest and bug log.
4	Posts, Comments & Notifications	Post creation flow with image upload to Cloud Storage; Post Details with Comments and Join/Interested; organizer badges (RSO vs. individual); location-tagged threads pinning a board post onto the Explore map; Notifications screen; integration testing pass across both teams.
5	Stretch, Polish & Demo	Attempt a stretch goal only once the required app is stable (see Section 8); full-team playtest and bug fixing; finalize the written report/documentation and README with Member 5; rehearse the live demo; tag a GitHub release.

Running this closer to 4 weeks? Don't compress by rushing Week 1's GitHub setup — that's exactly the step a first-time-Git team can't afford to skip. If Week 4 slips, cut the stretch goal before cutting Week 5's polish and testing time. A smaller, finished, well-tested app beats a bigger, broken one on demo day.

6. Weekly Task Checklist, Per Member

The same five weeks as Section 5, broken down into concrete, checkable tasks for each person.

WEEK 1 — Setup & Foundations

Member 1 Leader	Member 2 Frontend Head	Member 3 Frontend	Member 4 Backend Head	Member 5 Backend + Docs
□ Create the GitHub repo, invite the team, set a branch-naming convention and a simple PR template □ Set up the shared Google Doc Member 5 will use for weekly reports □ Draft the Firestore schema (users, posts, boards, events, comments) on paper; circulate for sign-off □ Set the real start/end dates for each week and confirm the check-in schedule	□ Get the React Native toolchain running locally, then help the rest of the team get theirs running □ Set up the React Navigation skeleton (Stack + Bottom Tabs) with empty placeholder screens □ Walk the Figma file with Member 3; list every reusable component needed (card, button, chip, input) □ Complete the GitHub Skills "Introduction to GitHub" module	□ Complete the GitHub Skills "Introduction to GitHub" module □ Pull design tokens from Figma (colors, spacing, type scale) into one shared style file □ Confirm with Members 4/5 exactly what fields each screen needs from Firestore □ Practice a first small PR to get comfortable with the flow before Week 2's real work starts	□ Create the Firebase project; enable Authentication, Cloud Firestore, and Cloud Storage □ Set up the @tip.edu.ph email restriction; decide how a "verified guest" account works □ Draft first-pass Firestore security rules (tighten later, once features land) □ Complete the GitHub Skills "Introduction to GitHub" module	□ Complete the GitHub Skills "Introduction to GitHub" module □ Set up the README skeleton and repo issue labels (bug, feature, docs, blocked) □ Write the Week 1 entry in the shared report doc, summarizing what the team decided □ Help Member 4 confirm the "posts"/"comments" fields the front-end will need

WEEK 2 — Auth + Core Screens

Member 1	Member 2	Member 3	Member 4	Member 5
□ Check Week 1's schema against what got built; resolve any mismatches □ Run the weekly check-in □ Start drafting the mid-sprint status note for Sir De Los Reyes	□ Wire the four bottom tabs to real navigation (placeholder data is fine) □ Build out the shared component library agreed on in Week 1	□ Build the login/signup screen UI; connect it to Firebase Auth with Member 4 □ Wire the Context API session so the logged-in user is available app-wide	□ Finish Firebase Authentication end-to-end (restriction, guest path, error states) □ Stand up the "users" Firestore collection	□ Stand up the "boards" Firestore collection with seed/test data □ Log Week 2 progress in the shared report doc; start the test-case checklist

WEEK 3 — Full Loop: Explore, Feed & Boards

Member 1	Member 2	Member 3	Member 4	Member 5
□ Review the Explore map + geo-query approach with Member 4 before it's built out further □ Unblock whichever team is behind □ Run the first full-team playtest	□ Integrate React Native Maps on Explore; render pins from real Firestore data	□ Build the Feed (FlatList + pull-to-refresh) and the Boards list (join/member counts) □ Build Search with recent searches + suggested results	□ Implement the nearby-events geo-query (geohash-based) behind Explore and Search	□ Wire board membership (join/leave) logic □ Log playtest bugs into the tracker; Week 3 report doc entry

WEEK 4 — Posts, Comments & Notifications

Member 1	Member 2	Member 3	Member 4	Member 5
☐ Decide whether both Advanced features fit, or whether one should protect Week 5 ☐ Merge this week's front-end/back-end branches	☐ Build the post-creation UI (+ button, image picker) ☐ Build the Post Details screen layout	☐ Build Comments and Join/Interested on Post Details; build Notifications ☐ Add organizer badges (RSO vs. individual) to event/post cards	☐ Wire image upload to Firebase Cloud Storage (posts + profile photos) ☐ Implement the organizer-badge data behind the UI	☐ Wire location-tagged threads (pinning a board post onto the Explore map) ☐ Run an integration pass across front-end/back-end; Week 4 report doc entry

WEEK 5 — Stretch, Polish & Demo

Member 1	Member 2	Member 3	Member 4	Member 5
☐ Make the final call on which stretch goal (if any) gets attempted ☐ Run the last full-team playtest; rehearse the demo run-of-show	☐ Bug-fix and polish Explore/Feed/Boards UI ☐ Help rehearse the live demo	☐ Bug-fix and polish Profile/Post Details/Notifications UI ☐ Prep demo talking points for the front-end features	☐ Bug-fix backend/data issues found in the playtest ☐ If time allows, prototype the AI-recommendations stretch goal	☐ Finalize the README and the written report/documentation for Sir De Los Reyes ☐ Tag the GitHub release

7. Tools & Platforms

Need	Tool	Why
Framework	React Native (Expo-managed workflow)	Fastest path to a working app on five different laptops/phones, without native Android/iOS build tooling on day one. (<i>Assumption — see Section 9.</i>)
Language	JavaScript (ES6+)	Keeps the learning curve to one new thing — React Native — instead of two, in a 5-week window. (<i>Assumption — see Section 9.</i>)
Navigation	React Navigation (Stack + Bottom Tabs)	The standard, well-documented library for this exact Explore/Feed/Boards/Profile tab structure.
State management	React Context API + Hooks	No extra library to learn; enough for an app this size (session, active filters, cached posts).
Maps & location	React Native Maps + Expo Location	Renders the Explore map and reads device location for "what's near me" queries.
Backend (BaaS)	Firebase — Auth, Cloud Firestore, Cloud Storage	One hosted backend instead of a custom server to write and deploy — matches the proposal's stack.
Version control + board	GitHub + GitHub Projects	Free; a Kanban board lives directly inside the same repo as the code.
Learning Git/GitHub	GitHub Skills · "Introduction to GitHub"	Free and interactive — built for a first-time team; where Week 1's onboarding time should go.
Design	Figma	Already the source of truth for the UI, the app flow chart, and the technical-plan slides.
Shared report doc	Google Docs	Free real-time co-editing for the weekly reports Member 5 is coordinating.
Debugger	Expo Dev Tools / React Native Debugger	Inspect state, logs, and network calls — far easier to reason about than <code>console.log</code> alone once the app has five-plus screens.
Team chat	Discord (<i>assumed — confirm</i>)	Free voice + text, easy to set up for a five-person team.

8. Feature Set

Grounded in your proposal's four core features, sequenced by what has to exist before the next thing can be built.

FOUNDATION — Weeks 1–2

- User Auth — login/signup restricted to @tip.edu.ph, plus a verified-guest path **BOTH**
- Bottom-tab navigation shell — Explore / Feed / Boards / Profile **FRONTEND**
- Firestore schema live — users, boards collections **BACKEND**

CORE — Required

- **Explore Map with Identity-Based Filters** — live pins for nearby events/activity, filterable by Students, Community Members, Spontaneous Events, Search & Tags **BOTH**
- **Location-Adaptive Feed** — FlatList stream of posts/events that shifts as you move between areas **FRONTEND**
- **Campus-Scoped Community Boards & Niche Sub-Groups** — interest tags, geofenced radius, location-tagged threads **BOTH**
- Search — recent searches + suggested results **FRONTEND**
- Profile — view and edit **FRONTEND**
- Post Details — with Comments and Join/Interested **BOTH**

ENHANCED — Required

- **Verified Event Hosting & Organizer Attribution** — RSO vs. individual badges **BOTH**
- Notifications **BOTH**
- Post creation, with image upload to Cloud Storage **BOTH**

STRETCH — Attempt only once the required set is stable

- **AI Recommendations** — personalized board/event suggestions, from the React Native Strategy slide. The heaviest lift on this list — attempt it last, in Week 5, and only if ahead of schedule.
- A richer organizer-verification workflow, beyond a static badge

9. A Few Notes From Your Consultant

- **Small, frequent commits.** One feature = one branch = one PR, reviewed by at least one teammate before it merges into main. Nobody's used GitHub before, so treat Week 1's practice time as a real task, not an optional extra.
- **Lock the Firestore schema before Week 2 gets going.** Since front-end and back-end are split into two teams, both sides need to agree on what a "post" or "board" document looks like before building against it — otherwise you'll be reshaping data mid-sprint.
- **Guard your Firebase keys.** Don't commit them to the public repo — use a `.env` file (and add it to `.gitignore`) from Day 1.
- **Test on both Android and iOS** (or at minimum, Expo Go on two different phones). React Native mostly behaves the same on both, but maps, permissions, and image pickers are the most common place where they don't.
- **If Week 4 slips, cut a stretch goal before cutting Week 5's polish and testing time.** A smaller, finished, well-tested app beats a bigger, broken one on demo day.

Decisions I still need from you before Week 1 locks in

As your consultant, I didn't want to quietly assume these — flag any of them and I'll revise the plan.

- 1 **Expo vs. bare React Native.** I assumed the Expo-managed workflow, since it's the fastest way to get five first-time devices running and building without native tooling friction. If the course requires a native module Expo doesn't support, tell me and I'll rework the Week 1 setup steps around bare React Native instead.
- 2 **JavaScript vs. TypeScript.** Neither the proposal nor the prototype specifies one, so I planned around plain JavaScript to keep the learning curve to one new thing. Say the word if the team wants TypeScript — it's a bigger ask for four Git first-timers inside five weeks, but doable if you'd rather.
- 3 **Team chat tool.** I defaulted to Discord in Section 7 since it's free and common for a small student team. Swap in Messenger, Slack, or whatever group chat you already have going.
- 4 **Who actually fills Members 2–5.** I structured the plan as "Member 2 = Frontend Head, Member 3 = Frontend, Member 4 = Backend Head, Member 5 = Backend + Docs" based on roughly equal, front-end/back-end-balanced workloads. You know everyone's actual strengths — happy to re-shuffle if, say, two teammates are stronger on design and two are stronger on Firebase/logic.
- 5 **Scope of "AI Recommendations."** The React Native Strategy slide mentions it, but it isn't one of your proposal's four core features, so I've placed it as a Week 5 stretch goal rather than a required deliverable. Confirm that's right, or tell me if Sir De Los Reyes expects it as core.
- 6 **What "verified" actually means for an organizer badge.** The proposal calls for distinguishing official RSOs from individual students, but doesn't say how an org gets that verified status. I've scoped it as a simple flag/badge for now rather than a real verification workflow (e.g. an admin approval step) — flag if that's too thin for what's being graded.

Prepared as your project consultant for CS409, Group 2 · Vicinia · Formatted to sit alongside — and stay clearly distinct from — the CS 003 guidelines.